TP8 · Food Store en la feria

Base de Datos II · UTN · Ciclo 2026

Equipo: Moron / Menendez · Referencia: rama `NoSQL` · Trabajo: rama `sqlite`

1. Base de la tablet y selección de datos

La tablet consulta un catálogo local y registra borradores durante una feria sin conexión. Un resultado `SIMULADO_APROBADO` solamente indica que el borrador coincide con la instantánea: no confirma una venta ni reserva unidades. PostgreSQL conserva la autoridad sobre precios, existencias y pedidos definitivos.

Dato	Viaja	Uso en la tablet
`producto.id`, `nombre`	Sí	Identificar y mostrar el producto.
`precio`	Sí, como `precio_centavos` entero	Guardar el importe visto sin aritmética de coma flotante.
`stock`	Sí, como referencia	Advertir disponibilidad; nunca constituye una reserva.
`activo`, baja lógica	Sí	Disponibilidad y lápidas; distinguir baja de ausencia.
Fecha de copia	Sí	Informar antigüedad de la instantánea.
Usuarios, contraseñas, correo y datos de clientes	No	No son necesarios para el catálogo offline.
Pedidos confirmados y datos de auditoría	No	Permanecen bajo control del servidor.

El campo local `usuario` es un alias sintético de operador, sin información personal. Solo se usan productos y pedidos inventados. Las columnas del contrato móvil se adaptan al esquema de referencia: `activo` aporta disponibilidad y la marca de eliminación representa una baja lógica.

`movil/schema_sqlite.sql` activa las claves foráneas y define `producto_cache`, `pedido_borrador`, `item_borrador` y `cola_sync` como tablas STRICT. CHECK restringe importes, cantidades, pagos, indicadores y estados; las FK evitan ítems huérfanos. La tabla auxiliar `snapshot_servidor` recibe el CSV del servidor, incluyendo booleanos `t` y `f`.

El borrador completo se guarda mediante una transacción que incluye cabecera, ítems y solicitud. Si una operación falla, se revierte la unidad completa. `pruebas_sqlite.sql` muestra COMMIT, ROLLBACK, catálogo offline y errores intencionales C7; las pruebas automatizadas verifican que esos errores no dejan filas adicionales.

2. Instantánea, semáforo y casos

La vista `v_snapshot_movil` produce el contrato `id,nombre,precio_centavos,stock,disponible,eliminado,fecha_copia`. Se exporta desde PostgreSQL con `\copy` y se importa a `snapshot_servidor`. La recarga convierte `t/f` a `1/0` y conserva las lápidas en `producto_cache`; el catálogo filtra los productos dados de baja. La instantánea base fue exportada el 01/11/2026 a las 19:45:35 UTC; la fecha queda en el CSV y en la base entregada. La referencia no versiona el esquema original completo: PostgreSQL se probó sobre una fixture mínima sintética, no una restauración de la base real.

El semáforo compara todos los ítems de cada solicitud PENDIENTE. Primero rechaza inexistencia, baja, indisponibilidad o stock insuficiente; solo si no hay problemas rojos considera cambios de precio. Coincidencia total produce `SIMULADO_APROBADO`. Las solicitudes resueltas quedan fuera de las siguientes ejecuciones.

Comando de verificación local desde la raíz: `python -m unittest discover -s movil/tests -v`. Los comandos individuales, exportaciones y salidas reales se conservan en `docs/evidencia_casos.md` y `docs/evidencia_postgresql.md`.

Caso	Preparación y operación	Resultado que se verifica
C1	Consultar cache sin conexión PostgreSQL.	Catálogo local disponible.
C2	Crear borrador, subir precio en copia del servidor, exportar y evaluar. Aceptar precio y enviar UUID nuevo.	CONFLICTO; después SIMULADO_APROBADO.
C3	Pedir más unidades que el stock de la instantánea.	RECHAZADO.
C4	Dar de baja el producto después del borrador, exportar y recargar cache.	RECHAZADO; desaparece del catálogo y permanece su fila.
C5	Enviar borrador sin modificaciones del servidor.	SIMULADO_APROBADO.
C6	Repetir UUID y ejecutar nuevamente el semáforo.	Error de PK; cantidad de solicitudes y resultado previo conservados.
C7	Insertar cantidad cero y producto inexistente.	Errores CHECK y FK; conteos sin cambios.

En C2 el cambio aceptado se aplica al precio visto del ítem, sin sobrescribir el precio del servidor. La solicitud previa conserva su resultado; el reintento usa otro UUID. En C4 la lápida impide confundir un artículo eliminado con uno habilitado y mantiene la referencia del borrador.

C6 demuestra idempotencia de esta simulación local. En un sistema real, la clave idempotente se controla también en el servidor con una restricción única y una transacción que guarda la respuesta junto con la operación definitiva. La PK de una tablet no protege por sí sola contra reenvíos desde otros dispositivos.

La comparación PostgreSQL antes/después fue ejecutada tras la simulación: pedidos=1; producto 10, stock=10 y precio=4500.00; producto 15, stock=5 y precio=1200.00. Los valores permanecieron idénticos. La línea de base se tomó después de preparar los escenarios; los cambios deliberados del caso no se confunden con efectos de la sincronización.

3. Integración y mantenimiento

Mapa de los tres motores

Motor	Datos y función	Autoridad
PostgreSQL	Productos, precios, stock, clientes, pedidos y detalles; restricciones y transacciones.	Fuente de verdad operativa.
MongoDB, Unidad 7	`foodstore_nosql.pedidos`: documentos con `_id`, estado, cliente embebido, arreglo `lineas` y total; cinco ejemplos en `sql/consultas_mongodb.js`.	Proyección documental para consultas. No reemplaza las validaciones operativas.
SQLite, Unidad 8	Cache del catálogo, borradores, ítems y cola con resultados de simulación.	Autoridad sobre el borrador local; sus precios y stock son referencias.

Los identificadores permiten relacionar representaciones. El modelo MongoDB del repositorio favorece la lectura de un pedido completo; no existe aquí un servicio automático de replicación entre motores. La simulación SQLite tampoco registra ventas ni actualiza documentos MongoDB.

Agregado al plan de la Unidad 6

- **PostgreSQL:** responsable técnico propone respaldo diario con `pg_dump -Fc`, retención y copia protegida fuera del equipo. Cada semana restaura en una instancia descartable; compara conteos, restricciones, una muestra de pedidos y stock. Registra duración y antigüedad real del respaldo. Los scripts históricos necesitan revisión previa y no forman parte del arranque automático de TP8.

- **MongoDB:** si se conserva una proyección no reconstruible, respaldarla con su herramienta nativa y probar su restauración. Si deriva de PostgreSQL, documentar el procedimiento de reconstrucción y verificar los identificadores; el TP7 actual es una carga de ejemplo, no una réplica configurada.

- **SQLite:** el catálogo puede reconstruirse. Los borradores y la cola requieren copia consistente con la API de respaldo de SQLite o la aplicación cerrada; verificar `PRAGMA integrity_check` y `foreign_key_check` al restaurar. Guardar solo datos necesarios, sin secretos ni datos personales reales.

- **Restauración antigua del servidor:** detener envíos, conservar los borradores y renovar la instantánea completa. Invalidar la suposición de vigencia de resultados anteriores, reconciliar contra el servidor restaurado y revisar antes de generar solicitudes nuevas. No reproducir ventas automáticamente. Una implementación real necesita identificar la época de recuperación del servidor.

- **Conflictos:** propuesta operativa: el encargado de feria revisa CONFLICTO/RECHAZADO al recuperar conexión y antes de cada cierre diario. El cliente debe aceptar el nuevo precio o modificar el borrador. El responsable técnico revisa casos persistentes diariamente; queda registro del motivo y se usa otro UUID al reenviar.

Estas frecuencias y responsabilidades son un plan propuesto, no tareas programadas ni controles desplegados.

4. Revisión de scripts con IA

Revisión realizada por **Codex**, por lectura del repositorio y pruebas aisladas. No se ejecutaron scripts de roles ni restauraciones sobre bases existentes. El requisito literal de repetir la revisión con OpenCode en agente Plan queda identificado como una actividad pendiente del equipo.

Alcance leído: `db/preparar_food_store.sql`, `tp_fnbc_control_lote.sql`, `tp_desnormalizacion_top_categorias.sql`, `maintenance.sql`, `health_check.sql`, `backup_foodstore.ps1`, `restore_test.ps1`; `sql/roles.sql`, `usuario_anon.sql`, `simulacro_incidente.sql`, `auditoria_test.sql`, `consultas_postgresql.sql` y `consultas_mongodb.js`. También se revisan los nuevos scripts móviles mediante sus pruebas. Los hallazgos históricos aceptados se registran sin modificar esas unidades en esta entrega.

Hallazgo	Unidad / decisión	Fundamento y comprobación
«Eliminar `subtotal` porque se desincroniza de cantidad y precio».	U4 · DESCARTADO	Es GENERATED ALWAYS STORED: se recalcula y no acepta un valor arbitrario. PostgreSQL: 2×4500 produce 9000; cambiar cantidad a 3 produce 13500; sobrescribir subtotal falla con SQLSTATE 428C9. Véase `evidencia_revision.md`.
`fn_autenticar` acepta cualquier contraseña para un correo existente.	U5 · ACEPTADO	La función no usa el argumento contraseña; las claves de prueba A y B producen true para el mismo cliente sintético; un correo inexistente produce false. No es autenticación válida. Se documenta para corrección separada; no se modifica seguridad.
Tratar `t/f` del CSV como enteros al importar.	U8 · CORREGIDO	Se conserva el texto en staging y se transforma explícitamente al cargar la cache. Pruebas con ambos valores y consulta de filas habilitadas/dadas de baja.
La documentación U4 afirma que no cumplir FNBC implica no cumplir 3FN.	U4 · ACEPTADO	Sus tres atributos son primos; la conclusión sobre 3FN no se deduce de la violación FNBC. La comprobación de equivalencia no prueba esta afirmación teórica. Pendiente corregir el comentario histórico.
Scripts históricos incluyen credenciales literales y el restaurador no controla todos los códigos nativos.	U5/U6 · ACEPTADO	Lectura directa, sin copiar valores ni ejecutar esos scripts. El anuncio de éxito no prueba que `pg_restore` y las consultas hayan terminado bien. Pendiente intervención separada.

El descarte de `subtotal` se limita al riesgo de desincronización de esa columna. No prueba por sí solo la corrección de todos los triggers del resumen por categoría. Del mismo modo, las pruebas nuevas no certifican los resultados históricos de las unidades anteriores.

La evidencia detallada distingue comprobaciones en PostgreSQL, SQLite y revisión estática. No se copian valores de credenciales en el informe, en las pruebas ni en el paquete TP8.

5. Trazabilidad, uso de IA y defensa

Los archivos `docs/kiro/requirements.md`, `design.md` y `tasks.md` describen los requisitos y el diseño del TP8. Los cuatro requisitos EARS relacionan el comportamiento con los casos de aceptación. Su existencia no equivale a haber ejecutado Kiro: se prepararon como archivos del proyecto mediante Codex.

La implementación usa SQLite y Python estándar para reproducir la simulación y ejecutar pruebas; no necesita aplicación móvil, interfaz ni conexión continua. `docs/evidencia_tdd.md` conserva el ciclo de pruebas. El resultado de cada caso y sus límites se consulta en los documentos de evidencia, no se infiere de una captura.

`docs/bitacora_ia.md` registra la asistencia real: implementación basada en el esquema existente, decisiones aceptadas y descartes fundamentados. `docs/DUIA.md` declara la herramienta usada y delimita las actividades que el equipo debe revisar y realizar personalmente; no sustituye una firma ni un formulario oficial de la cátedra que no fue suministrado.

Preparación de la defensa

Guion de diez minutos en `docs/defensa_tp8.md`: explicar el mapa de motores, ejecutar un cambio de precio y la resolución con UUID nuevo, y justificar un hallazgo de revisión. Cada integrante debe poder explicar claves foráneas, transacciones, centavos, lápidas, prioridad roja y límite de la idempotencia local.

Actividades personales pendientes

1. Leer los scripts y resultados, reproducirlos y poder explicarlos individualmente.

2. Realizar el flujo requerido con Kiro y OpenCode Plan, registrando sus resultados reales y las diferencias encontradas. No atribuir esta ejecución a esas herramientas.

3. Completar comisión, identificación y formulario oficial DUIA si la cátedra exige un formato específico; revisar y firmar personalmente.

4. Hacer el ensayo de defensa en clase y entregar por el campus. El ZIP no demuestra esas actividades.

El paquete incluye scripts, instantánea sintética, base SQLite, informe y documentación de TP8. No incorpora los scripts históricos con secretos detectados. La rama local conserva el historial de referencia para revisión; no se publica ni se modifica el servidor del proyecto como parte de la simulación.